

Extra (unstable) packets for time extension

Extra packets

SPIDR firmware with version 0.2 or later appends additional 64-bit words ("packets") to every outgoing Ethernet chunk.

These extra packets provide:

1. **Packet Generation Timestamp:** when the firmware decided to “close” the chunk and send it out. You should not need this packet for anything. We use it to debug the SPIDR firmware.
2. **Minimum Event Timestamp:** the earliest timestamp of all timed events in that chunk. This includes all events: pixel hits, TDCs, and control packets (like heartbeats, shutter open, etc.).
3. **Maximum Event Timestamp:** the latest timestamp of all timed events in that chunk. This includes all events: pixel hits, TDCs, and control packets (like heartbeats, shutter open, etc.).

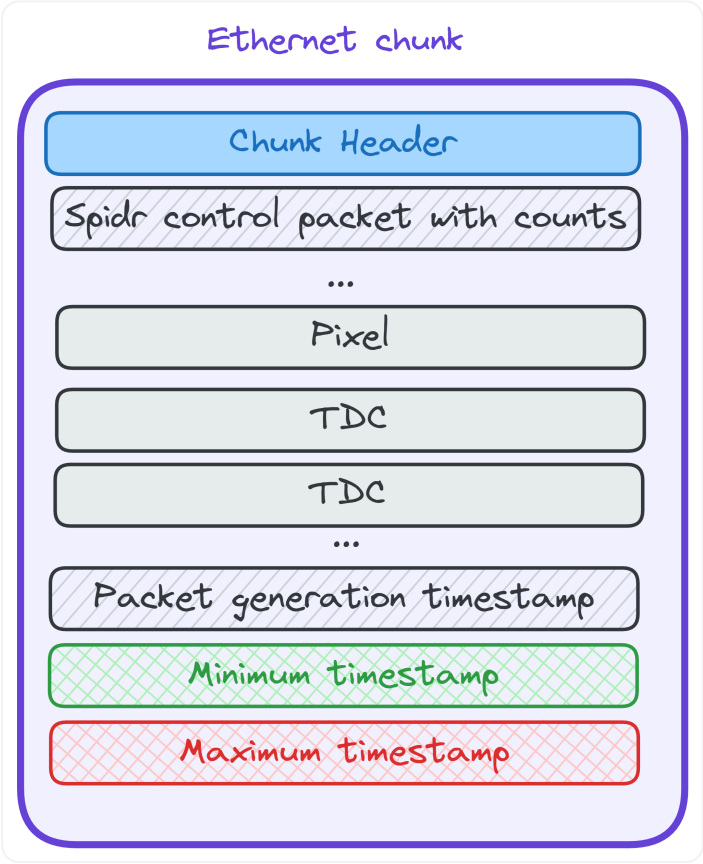
These extra timestamps can help you to:

- **Sort** hits faster when reconstructing data on the PC side.
- **Extend** the timestamps by up to 325 days.

Because these features are **experimental**, the details below may change in future firmware releases and/or we might decide to use a different approach to speed up sorting.

How to decode the extra packets:

The extra timestamp packets are added at the end of each Ethernet chunk, so the chunks look like:



The extra timestamp packets have all the same header and structure as listed below. **You can identify them only by their position in the chunk.**

Since you only need the minimum and maximum timestamps, I will detail them in the table below:

63-56 bit	55 bit	54 bit	53-0 bits
Header 0x51 (TPX3) 0x21 (MPX3)	Error flag For Debug only Should be 0	Overflow For Debug only Indicates if an overflow happened when calculation	Timestamp (1.5625 ns units) You can use this to extend all timestamps in the chunk.

Extending time using the minimum timestamp

You can extend all the timestamps from the chunk using the formula (here implemented in Rust):

```
1  pub fn extend_timestamp(  
2      timestamp: u64,  
3      minimum_timestamp: u64,  
4      n_bits: u64,  
5  ) → u64 {  
6      // Get the maximum value that the timestamp can have  
7      let bit_mask = (1 << n_bits) - 1;  
8  
9      // Calculate the least significant bits while ensuring  
10     // the value remains greater than `minimum_timestamp`.  
11     // If the result is negative, a wraparound has occurred,  
12     // and we should take the value after it.  
13     //  
14     // The integer subtraction accounts for this, and we then  
15     // extract only the significant bits using the mask.  
16     //  
17     // Rust enforces strict underflow checks, so we must  
    explicitly  
18     // allow wrapping. In other languages, this operation can be  
    written as:  
19     // (timestamp - minimum_timestamp) & bit_mask  
20     let delta_t = timestamp.wrapping_sub(minimum_timestamp) &  
        bit_mask;  
21  
22     minimum_timestamp + delta_t  
23 }
```

For example, you can decode all timestamps at 30 bits (which is the minimum precision in all packets) and use the formula above to extend any of them.

You can also use minimum/maximum timestamps to decide if a whole chunk is in a "frame" or perform other optimization in your data pipeline that concert processing the elements in a bulk.